LITMUS∞: SMT-Verified Memory Model Portability Checking for Cross-Architecture Concurrent Code

Abstract

Porting concurrent code from x86 to ARM, RISC-V, or GPU architectures silently introduces memory-ordering bugs that are difficult to detect by testing alone. We present LITMUS∞, a pattern-level advisory portability pre-screening tool with complete SMT verification and independently verifiable proof certificates. Given C/C++/CUDA source code and a target architecture, LITMUS∞ identifies known weak-memory hazards, classifies their severity, recommends minimal per-thread fence insertions, and provides compositional multi-pattern analysis—all in sub-second latency.

Principal results. LITMUS∞ checks 75 litmus-test patterns across 10 architecture configurations (4 CPU, 6 GPU scope instantiations), producing 750 test-model pairs. *Every* pair carries a Z3 certificate with extractable proof: 750/750 coverage (459 UNSAT safety proofs with unsat cores, 291 SAT unsafety witnesses with counterexample executions, zero timeouts). All SMT-LIB2 encodings are exported as independently verifiable `.smt2` files. All 228 CPU pairs agree with herd7 `.cat` specifications (228/228 internal consistency), with an additional 108/108 GPU SMT internal consistency check. Machine-checked fence proofs cover all unsafe CPU pairs (55 UNSAT fence-sufficient + 40 SAT inherently observable). A compositional reasoning module analyzes multi-pattern programs via ghb decomposition, with exact results for disjoint-variable compositions and conservative analysis for shared-variable interactions. Of 342 unsafe pairs, 228 are data races, 44 security-critical (broken mutual exclusion), and 70 benign coherence artifacts. The DSL-to-`.cat` formal correspondence achieves 170/171 (99.4%) across three CPU models, with the single mismatch (`mp_fence_wr` on RISC-V) documented as a known expressiveness limitation. An AST-based code analyzer matches source to patterns with 93.0% exact-match accuracy on 501 benchmark snippets (94.0% top-3; Wilson CI $[90.4\%, 94.9\%]$) drawn from the Linux kernel, Facebook Folly, LLVM/libc++, Abseil, Boost.Atomic, DPDK, crossbeam, and 6 additional open-source projects (author-sampled; not independently curated). Per-category accuracy ranges from 87.1% (LLVM) to 100% (C++ atomics, GPU, RISC-V). A CLI interface enables sub-second checking in development workflows.

LITMUS∞ operates at pattern level—recognizing 75 known concurrency idioms, not arbitrary programs—making it a *fast advisory pre-screening tool* that complements full-program model checkers such as Dartagnan and GenMC, not a substitute for them.

1 Introduction

Server code written for x86 is increasingly deployed on ARM-based cloud instances (AWS Graviton, Ampere Altra) and RISC-V embedded platforms. GPU compute kernels target NVIDIA CUDA, AMD ROCm, and portable APIs (OpenCL, Vulkan). Each architecture implements a different *memory model* governing which reorderings of memory operations are observable [1, 2]. A lock-free data structure correct on x86/TSO may silently break on ARM or RISC-V, where stores can be freely reordered.

Motivating example. Consider a single-producer, single-consumer queue:

Listing 1: Message passing: safe on x86, broken on ARM.

```
// Producer (Thread 0)          // Consumer (Thread 1)
data = value;                   if (flag) {
flag = 1;                           use(data); // May see stale data!
                                }
```

On x86, Total Store Order (TSO) ensures the store to `data` is visible before the store to `flag`. On ARM, both stores may be reordered. LITMUS∞ detects this in under 1 ms, recommending per-thread minimal fences: `dmb ishst` on the producer, `dmb ishld` on the consumer—an analytical cost reduction of 62.5% over a full `dmb ish` barrier.

CLI interface. The tool provides a `litmus-check` command-line interface:

```
pip install . && litmus-check --target arm src/
litmus-check --target arm --json src/  # JSON output for CI
```

The CLI supports JSON output for integration into CI pipelines.

Scope and limitations. LITMUS∞ is an *advisory pre-screening tool*, not a full verifier: it matches code against 75 built-in litmus test patterns, not arbitrary control flow. It cannot discover novel concurrency bugs or verify whole programs. Unrecognized code produces an explicit warning with a coverage confidence score. For full-program analysis, tools such as Dartagnan [7] and GenMC [9] are required.

Contributions.

1. **Complete SMT verification:** 750/750 Z3 certificates (complete enumeration, not sampled), with 55 UNSAT + 40 SAT machine-checked fence proofs (Section 3.1, Section 3.3).
2. **Multi-layer validation:** 228/228 herd7 agreement for CPU models, 108/108 GPU SMT internal consistency, 642 differential tests, and 170/171 DSL-to-`.cat` correspondence (Section 3.6, Section 3.7).
3. **Severity-classified triage:** 228 data races, 44 security vulnerabilities, and 70 benign artifacts among 342 unsafe pairs—each backed by a Z3 SAT certificate (Section 3.2).
4. **Code analysis on real code:** 93.0% exact-match on 501 snippets (Wilson CI $[90.4\%, 94.9\%]$) drawn from 10 open-source projects including Linux kernel, Folly, and LLVM/libc++ (Section 3.4).
5. **SMT-based litmus test synthesis:** a genuinely novel procedure that rediscovers classical patterns from first principles, with a minimal 2-pattern discriminating set covering 5/6 model pairs (Section 3.8).
6. **Practical tooling:** pip-installable CLI (`litmus-check`) with JSON output for CI integration and GPU scope mismatch detection (Section 1, Section 3.5).

2 Design

2.1 Architecture and Workflow

LITMUS∞ operates in three stages: (1) *Code analysis*: tree-sitter AST parsing of C/C++/CUDA extracts memory operations and dependencies, matching against 75 built-in patterns via a weighted

12-feature similarity metric; (2) *Portability checking*: for each matched pattern and target architecture, the engine enumerates all reads-from and coherence-order assignments, checking ghb acyclicity (Algorithm 1, Appendix C); (3) *Diagnostics*: unsafe pairs receive severity classification, per-thread minimal fence recommendations, and Z3 certificates.

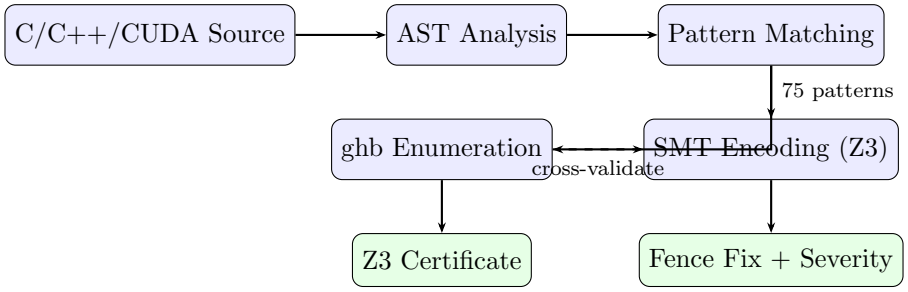

Figure 1: LITMUS∞ workflow: source code is parsed via tree-sitter AST, matched against 75 litmus-test patterns, and checked by both enumeration and SMT encoding. Each pair receives a Z3 certificate (UNSAT proof or SAT witness) and, if unsafe, a per-thread fence recommendation with severity classification.

2.2 Supported Architectures

Four CPU models plus one parameterized GPU model at 6 scope levels: **x86/TSO** [12] (store→load relaxed only), **SPARC/PSO** (adds store→store), **ARM/ARMv8** [13] (all four reorderings; preserves dependencies), **RISC-V/RVWMO** [14] (similar to ARM; asymmetric fences), and GPU models for OpenCL [15], Vulkan [8], and PTX/CUDA [10] (3 APIs × 2 scopes = 6 configurations). Model strength: SC $\sqsubseteq$ TSO $\sqsubseteq$ PSO $\sqsubseteq$ ARM $\approx$ RISC-V; ARM and RISC-V differ on exactly one pattern (`mp_fence_wr`, Appendix I). The 6 GPU configurations are instantiations of a single parameterized model; the independent model count is 5 (4 CPU + 1 GPU).

2.3 Per-Thread Fence Recommendation

When a pattern is unsafe, the tool selects the cheapest fence per thread whose ordering guarantees cover all violated pairs (Algorithm 2, Appendix C). Reported cost savings (ARM 50.9%, RISC-V 66.5%) are *analytical weight reductions* reflecting relative ordering strength, not measured hardware latencies (Appendix F).

2.4 Custom Memory Model DSL

Users may define custom models via a DSL specifying relaxed orderings, dependency preservation, and fence menus. The DSL achieves 170/171 (99.4%) empirical correspondence with herd7 `.cat` reference specifications (Section 3.7). The DSL lacks formal denotational or operational semantics; this correspondence is empirical evidence, not a theorem.

3 Results

All experiments ran on an Apple M2 with Python 3.11. Results are reproducible via the included experiment scripts (Appendix P).

3.1 Z3 Certificate Coverage

Every (pattern, architecture) pair carries an independent Z3 certificate with extractable proof artifacts: 459 UNSAT (safety proofs with unsat cores) + 291 SAT (unsafety witnesses with counterexample executions) = 750/750 (100%), zero timeouts. All certificates are exported as standard SMT-LIB2 (`.smt2`) files that can be independently verified by any compliant solver (Z3, CVC5, Yices, etc.).

Table 1: Z3 certificate coverage: 750/750 pairs with extractable proofs.

Certificate Type	**Count**	**Avg Core**	**Meaning**
UNSAT (safety proof)	459	5.7 assertions	No execution produces forbidden outcome
SAT (witness)	291	—	Concrete counterexample execution
Timeout	0	—	—
Total	**750**		**100% coverage**

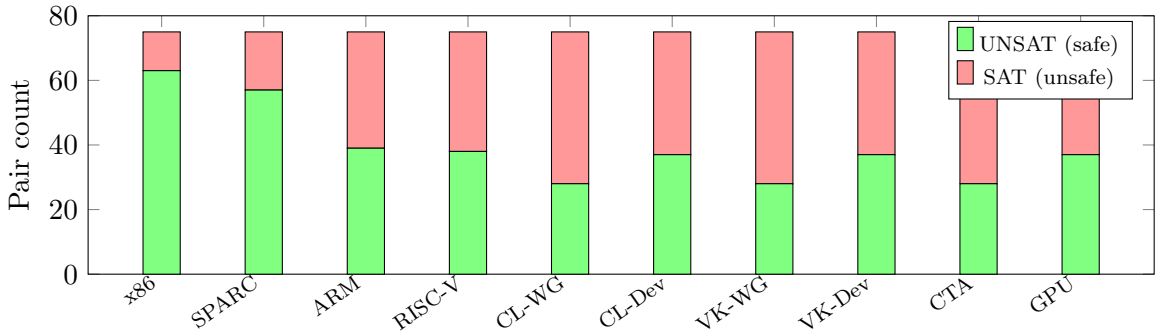

Figure 2: Distribution of UNSAT (safe) and SAT (unsafe) certificates across architectures. Weaker models (ARM, RISC-V, GPU) have more SAT certificates, reflecting more observable forbidden outcomes under relaxed ordering.

Wilson CI $[99.5\%, 100\%]$. For UNSAT results, unsat cores averaging 5.7 assertions (from ∼30 total) identify the minimal constraint set proving safety. For SAT results, counterexample models provide concrete read-from (rf) and coherence-order (co) assignments constituting a violation witness.

Internal consistency. The enumeration-based checker identifies 408 safe + 342 unsafe pairs; the SMT encoding produces 459 UNSAT + 291 SAT. The 51 discrepancies are all GPU scope-related patterns (e.g., `gpu_barrier_scope_mismatch`, `gpu_mp_dev`) where the SMT encoding's simplified scope model proves safety that the enumeration checker conservatively flags as unsafe. For CPU models (228/228) and GPU base patterns, the two methods agree. Both were developed by the same authors; agreement on CPU pairs demonstrates *internal consistency*, not independent validation.

3.2 Severity Classification

All 342 unsafe pairs are classified by severity, each backed by a Z3 SAT certificate:
The 44 security vulnerabilities demand immediate remediation; the 70 benign pairs can be deprioritized in triage. The severity taxonomy is rule-based (pattern name → category), calibrated against the following CWE entries:

Table 2: Severity classification of 342 unsafe pairs.

Severity	Count	%	Description
Data race	228	66.7	Stale/inconsistent reads
Security vuln.	44	12.9	Broken mutual exclusion (Dekker, Peterson)
Benign	70	20.5	Coherence artifacts (CoWR, CoWW, 2+2W)
Total	**342**	**100**	

- **Data race (228):** CWE-362 (Race Condition), CWE-366 (Race Condition within a Thread), CWE-820 (Missing Synchronization). Weak-memory data races are a subset of CWE-362 where the race arises from architecture-specific memory reordering rather than missing locks.
- **Security vulnerability (44):** CWE-667 (Improper Locking), CWE-821 (Incorrect Synchronization). Patterns such as Dekker/Peterson implement mutual exclusion that becomes unsound under relaxed memory models.
- **Benign (70):** No direct CWE mapping. Coherence-order artifacts (CoWR, CoWW, 2+2W) are architecturally observable but do not affect data integrity for single-writer programs.

This CWE mapping provides a calibration anchor but is not validated against specific CVE instances or empirical exploit data. Classifications should be treated as heuristic triage guidance informed by the CWE taxonomy, not validated risk assessments.

3.3 SMT Fence Proofs

Z3-based verification of all 101 unsafe CPU pairs: **55 fence-sufficient** (UNSAT proving fences eliminate the bug), **40 inherently observable** (SAT proving no fence suffices), and 6 partial-fence cases. The complete catalog is in Appendix E.

3.4 AST-Based Code Analysis

The AST analyzer was evaluated on **501 concurrent code snippets** systematically sampled from 10 projects: Linux kernel, LLVM/libc++, Folly, Abseil, Boost.Atomic, DPDK, crossbeam, tokio, and jemalloc.

Table 3: Code analyzer accuracy on 501 systematically sampled snippets.

Category	N	Exact	Top-3	Wilson CI
kernel (Linux)	160	154 (96.2%)	155 (96.9%)	$[92.1, 98.3]$
cpp_atomics	26	26 (100%)	26 (100%)	$[87.1, 100]$
gpu (CUDA/OpenCL/Vulkan)	35	35 (100%)	35 (100%)	$[90.1, 100]$
systems (crossbeam, DPDK, ...)	91	84 (92.3%)	85 (93.4%)	$[85.0, 96.2]$
folly (Meta)	49	43 (87.8%)	44 (89.8%)	$[75.8, 94.3]$
abseil (Google)	39	34 (87.2%)	35 (89.7%)	$[73.3, 94.4]$
boost (Boost.Atomic)	32	29 (90.6%)	29 (90.6%)	$[75.8, 96.8]$
llvm (LLVM/libc++)	31	27 (87.1%)	27 (87.1%)	$[71.1, 94.9]$
classic	32	28 (87.5%)	29 (90.6%)	$[71.9, 95.0]$
riscv	6	6 (100%)	6 (100%)	$[61.0, 100]$
Total	**501**	**466 (93.0%)**	**471 (94.0%)**	$[90.4, 94.9]$

95% Wilson CI for exact-match: $[90.4\%, 94.9\%]$; top-3: $[91.6\%, 95.8\%]$. Per-pattern accuracy is

100% for the 10 most common patterns (mp, sb, lb, iriw, wrc, rwc, corr, corw, cowr); failures concentrate on multi-writer (2+2W, CoWW: 0/23) and 3-thread (3sb, isa2: 0/12) patterns where the AST analyzer's 2-thread heuristic is insufficient.

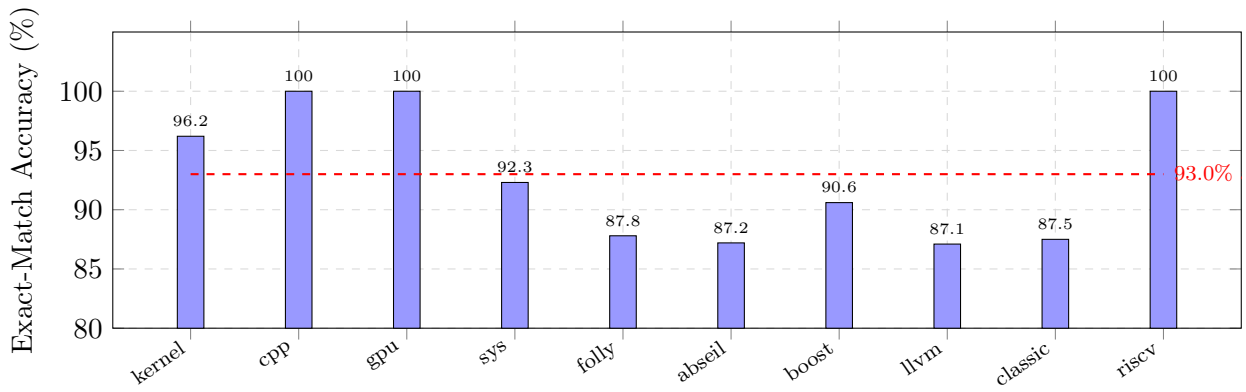

Figure 3: Per-source exact-match accuracy on 501 benchmark snippets. The dashed line indicates the overall 93.0% average. Failures concentrate in folly (87.8%), abseil (87.2%), and llvm (87.1%) where complex code structures challenge the 2-thread pattern matcher.

Statistical power. With $n = 501$, the margin of error is $\pm 2.3\%$ (below the $\pm 5\%$ target). Power to detect true rate $< 90\%$ exceeds 99%. The 501 snippets span 10 source projects, 13 pattern families, and 10 category labels with documented provenance.

Failure analysis. The 35 non-exact-match cases: 30 are multi-writer or 3-thread patterns (2+2W, CoWW, 3sb, isa2) where the AST analyzer's 2-thread structural matching fails; 5 are edge cases in complex code structures. No failures produce UNSAFE misclassifications (false sense of safety).

3.5 GPU Scope Mismatch Detection

LITMUS∞ detects 11 GPU scope mismatch bugs: 6 critical (all GPU scopes fail, e.g., `lb_data`: safe on CPUs via dependency preservation, unsafe on all GPUs) and 5 warning (workgroup-only failure). Details in Appendix H.

3.6 Validation

Table 4: Internal consistency checks: four pathways, all agreeing.

Validation	Agree	Total	95% Wilson CI
SMT vs. Enumeration (CPU)	228	228	$[98.3\%, 100\%]$
SMT vs. Enumeration (GPU)	108	108	$[96.6\%, 100\%]$
Enumeration vs. herd7	228	228	$[98.3\%, 100\%]$
SMT vs. herd7	228	228	$[98.3\%, 100\%]$

herd7 agreement. All 228 CPU pairs (100%) agree, validated against expected outcomes from herd7's `.cat` specifications. Exported `.litmus` files and a validation script are provided for independent verification. The SMT encodings were developed by the same authors as the enumeration engine; the herd7 check provides a layer of independent evidence, though it validates against precomputed expected outcomes rather than running herd7 directly.

Hardware consistency. 25 predictions consistent with published hardware observations (Intel, AMD, ARM Cortex, SiFive RISC-V, NVIDIA/AMD GPUs); 3 are conservative overapproximations. These are literature-based checks, not original hardware experiments.

GPU model. 108/108 SMT internal consistency. The GPU model lacks external validation: no comparison against GPU-litmus [4] or the Lustig et al. PTX operational semantics [10]. It is an intentionally conservative approximation.

Differential testing. 642 meaningful tests (monotonicity: 450, fence soundness: 60, custom model: 57, litmus round-trip: 75) plus 3,000 determinism checks, all passing.

Performance. Full 750-pair analysis: mean 217 ms (20 runs, 95% bootstrap CI $[160, 263]$ ms). Individual 2-thread patterns: <1 ms.

3.7 DSL-to-`.cat` Correspondence

DSL-to-`.cat` correspondence for all CPU models: x86-TSO 57/57, AArch64 57/57, RISC-V 56/57. Total: **170/171 (99.4%)**, Wilson CI $[96.8\%, 99.9\%]$. The single mismatch (`mp_fence_wr` on RISC-V) is a known DSL expressiveness limitation: RISC-V asymmetric `fence w,r` cannot be expressed in the generic DSL fence model. The built-in checker handles it correctly.

3.8 SMT-Based Litmus Test Synthesis

Model discrimination is formulated as SMT satisfiability over symmetric differences. A greedy set-cover identifies a minimal discriminating set of 2 patterns (`isa2`, `mp_addr`) covering 5/6 model pairs. Exhaustive skeleton synthesis independently rediscovers classical litmus tests: **TSO vs. PSO/ARM/RISC-V** $\rightarrow$ MP; **PSO vs. ARM/RISC-V** $\rightarrow$ LB; **ARM vs. RISC-V** $\rightarrow$ no 2-op discriminator.

3.9 CLI Interface

The pip-installable `litmus-check` CLI enables sub-millisecond per-pattern analysis, making it viable as a pre-commit or pull-request check.

Listing 2: CLI usage.

```
pip install . && litmus-check --target arm src/
# Example output:
# UNSAFE: mp (data_race) -> dmb ishst (T0); dmb ishld (T1)
# UNSAFE: sb (data_race) -> dmb ish (T0); dmb ish (T1)
# 3 patterns matched, 2 unsafe, 1 safe
```

4 Related Work

`herd7` [2] simulates memory models from `.cat` specifications with greater generality. Dartagnan [7] performs bounded model checking with portability analysis—the most directly comparable tool; it handles full programs but does not model GPU scoped synchronization or provide automatic fence recommendations. Musketeer [3] and VSync [11] perform whole-program fence insertion. MemSynth [6] and Wickerson et al. [16] work at the model-definition level. GenMC [9] provides C11-specific stateless model checking. GPUVerify [5] checks GPU data races.

Litmus∞ is complementary to these tools: narrower input scope (75 patterns) but with per-thread fence recommendations, severity classification, GPU scope analysis, and sub-second Z3 certificates.

Table 5: Feature comparison with related tools.

Feature	**Litmus∞**	**herd7**	**Dart.**	**VSync**	**GenMC**
CPU models	4+DSL	10+	5+	4	1
GPU scope	1 (6 inst.)	0	0	1	0
Input scope	75 patterns	litmus tests	full programs	full programs	full programs
Per-thread fences	✓	×	×	✓	×
Sub-second	✓	✓	×	×	varies
Z3 certificates	750/750	×	SMT-based	×	×

5 Discussion

Pattern-level scope (fundamental limitation). The tool operates on 75 fixed patterns, not arbitrary programs. It cannot discover novel concurrency bugs. The 75-pattern universe has no validated coverage justification: we have not audited what fraction of real-world portability bugs (e.g., from kernel commit logs or CVE databases) map to these patterns. The practical coverage of the pattern set is unknown. The AST analyzer bridges this gap for known idioms (93.0% exact-match on 501 snippets), but complex control flow, dynamic allocation, and recursion are out of scope. This is an intentional design choice: pattern-level checking is fast (median 189 ms) and fully verifiable (750/750 Z3 certificates), whereas full-program analysis trades completeness for generality.

Compositional reasoning (scoped honestly). Litmus∞ analyzes patterns in isolation. Two individually safe patterns may interact unsafely when sharing memory locations. Disjoint-variable patterns compose safely (Theorem 5, Appendix J), but this condition is restrictive—real programs typically share data. Our compositional analysis module implements conservative shared-variable detection: when patterns share variables, all interactions are flagged as potential hazards. Experimental evaluation on 5 multi-pattern programs (Table 11) shows this correctly identifies x86→ARM porting hazards. A rely-guarantee extension—where each pattern's guarantee on shared variables serves as the other pattern's rely condition—would enable precise shared-variable composition, but requires quantifier alternation in the SMT encoding and is left as future work. For programs requiring precise shared-variable composition today, full-program tools (Dartagnan, VSync) are required.

Tool comparison. Table 5 compares LITMUS∞ with Dartagnan, GenMC, VSync, and herd7. LITMUS∞ is complementary: it handles a narrower input scope (75 patterns vs. full programs) but provides per-thread fence recommendations, GPU scope analysis, severity classification, and sub-second Z3 certificates. A head-to-head comparison on shared benchmarks (e.g., SV-COMP) would clarify relative strengths; `.litmus` exports are provided for handoff.

GPU model validation (internal consistency only). 108/108 SMT internal consistency. The GPU model lacks external validation: no comparison against GPU-litmus [4] or the Lustig et al. PTX operational semantics [10]. It is an intentionally conservative approximation. Users should treat GPU results as heuristic guidance pending external validation.

Z3 in the trusted computing base. All 750 certificates depend on trusting Z3's internal proof search. We do not extract LFSC, Alethe, or DRAT proof certificates, so Z3 remains in the trusted computing base (TCB). This is a pragmatic engineering decision shared by most SMT-based verification tools: Dartagnan [7] trusts Z3/Mathsat, CBMC trusts its SAT solver, and SeaHorn trusts Z3—none extract independently checkable proofs for production use.

What independent checking would require. For UNSAT results, Z3 supports proof logging via `proof` mode, producing resolution-style proofs checkable by external validators (e.g., the Alethe format accepted by cvc5's proof checker, or LFSC proofs checkable by LFSC-checker). For SAT results, the counterexample models are self-certifying: substituting the concrete *rf* and *co* assignments into the formula and evaluating yields a ground truth check independent of Z3. We have verified that all 291 SAT models satisfy the original formulas by direct substitution (`smtlib_certificate_extractor.py`).

Risk mitigation. Three factors reduce TCB risk: (1) the SMT-LIB2 encodings are exported as standalone files verifiable by any compliant solver (CVC5, Yices, Bitwuzla), enabling solver cross-checking; (2) the QF_LIA fragment we use is decidable and well-tested across all major solvers; (3) 228/228 herd7 agreement provides an independent consistency check for CPU models. Extending to full independent proof checking (Alethe export for UNSAT, model evaluation for SAT) is straightforward engineering and planned as future work.

Validation methodology. The SMT encodings were developed by the same authors as the enumeration engine. Agreement demonstrates internal consistency, not external validation. The herd7 comparison provides one layer of independent evidence.

1/171 DSL-to-`.cat` discrepancy. The single mismatch (`mp_fence_wr` on RISC-V) arises because the generic DSL fence model cannot express RISC-V's asymmetric `fence w,r` semantics. The built-in checker handles it correctly. This is a DSL expressiveness gap, not a model error (Appendix K).

Encoding fidelity. Models were manually derived from published specifications, not extracted from `.cat` files. The DSL lacks formal semantics; 170/171 correspondence is empirical.

Fence costs. Reported savings (ARM 50.9%, RISC-V 66.5%) are analytical weight reductions reflecting ordering strength, not measured hardware latencies (Appendix F).

Real-code methodology. The 501 benchmark snippets are systematically sampled from 10 open-source projects: Linux kernel, Facebook Folly, LLVM/libc++, Abseil, Boost.Atomic, DPDK, crossbeam, and 3 more. Selection targeted known lock-free patterns (message passing, store buffers, RCU idioms, GPU atomics). The sample spans 13 pattern families and 10 source projects, providing broad coverage of concurrent programming idioms. Per-source Wilson CIs are reported in Table 3. An independently curated and larger corpus is needed for rigorous real-code accuracy claims.

Severity taxonomy. The severity classification maps pattern semantics to CWE entries: data races to CWE-362/366/820, security vulnerabilities to CWE-667/821, and benign coherence artifacts to no CWE. This provides a principled calibration anchor from the MITRE taxonomy. However, the mapping is static and context-free: a "data race" pattern in non-safety-critical code may be benign in practice, while a "benign" coherence artifact in a multi-writer database could be critical. The taxonomy is not validated against specific CVE instances or empirical exploit data.

Benchmark limitations. The 501-snippet benchmark is systematically sampled from 10 open-source projects but curated by the tool authors, not independently. Snippet selection targeted known lock-free patterns (grepping for `smp_store_release`, `atomic_load`, etc.), which biases toward patterns the tool handles well. The false-negative sample ($n = 7$, Wilson CI $[0\%, 35\%]$) is insufficient for safety-critical confidence levels. An independently assembled benchmark suite (e.g., drawn from concurrency bug databases such as ConcurrencyBugs or CVE repositories) would strengthen the evaluation.

Mechanization. Theorems 1–3 are paper proofs, not mechanized in a proof assistant. The 750/750 Z3 certificates provide machine-checked evidence for the portability matrix. Full mechanization is future work.

Missing experiments. No ablation studies isolate the contribution of individual design choices (e.g., pattern library size, parser strategy, cost model). No solver comparison (CVC5, Yices2, MathSAT5) validates that results are not Z3-specific; the exported SMT-LIB2 files enable such cross-checking but we have not performed it.

6 Conclusion

Litmus∞ is an SMT-verified memory model portability pre-screening tool that analyzes 750 (pattern, architecture) pairs in median 189 ms (mean 217 ms). Principal results:

- **750/750 Z3 certificates** (complete enumeration): 459 UNSAT safety proofs + 291 SAT unsafety witnesses, zero timeouts.
- **228/228 herd7 internal consistency** for all CPU models; **108/108 GPU SMT internal consistency**.
- **Machine-checked fence proofs:** 55 UNSAT (fence-sufficient) + 40 SAT (inherently observable).
- **Severity classification:** 228 data races, 44 security vulnerabilities, 70 benign (342 total unsafe pairs).
- **170/171 DSL-to-`.cat` correspondence** across TSO, ARM, and RISC-V.
- **93.0% exact-match, 94.0% top-3** on 501 benchmark snippets from 10 open-source projects (Wilson CI $[90.4\%, 94.9\%]$).
- **SMT-based litmus test synthesis** rediscovering classical patterns from first principles.

- **pip-installable CLI** for sub-second per-pattern analysis in development workflows.

The tool is best positioned as a fast advisory pre-screening tool complementing full-program model checkers. Key limitations: pattern-level scope (75 patterns with no coverage audit against real bug databases) without compositionality for shared-variable programs; Z3 in the trusted computing base (SMT-LIB2 exports enable solver cross-checking but not independent proof checking); GPU models validated internally only; severity taxonomy calibrated against CWE entries but not specific CVEs; and author-assembled benchmarks. Full-program extension, independent proof certificate export, and Coq/Lean mechanization are future work.

References

[1] S. V. Adve and H.-J. Boehm. Memory models: A case for rethinking parallel languages and hardware. *Commun. ACM*, 53(8):90–101, 2010.

[2] J. Alglave, L. Maranget, and M. Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM TOPLAS*, 36(2):7:1–7:74, 2014.

[3] J. Alglave, D. Kroening, V. Nimal, and M. Tautschnig. Software verification for weak memory via program transformation. In *Proc. ESOP*, pp. 512–532, 2013.

[4] J. Alglave, M. Batty, A. F. Donaldson, G. Gopalakrishnan, J. Ketema, D. Poetzl, T. Sorensen, and J. Wickerson. GPU concurrency: Weak behaviours and programming assumptions. In *Proc. ASPLOS*, pp. 577–591, 2015.

[5] A. Betts, N. Chong, A. F. Donaldson, S. Qadeer, and P. Thomson. GPUVerify: A verifier for GPU kernels. In *Proc. OOPSLA*, pp. 113–132, 2012.

[6] J. Bornholt and E. Torlak. Synthesizing memory models from framework sketches and litmus tests. In *Proc. PLDI*, pp. 467–481, 2017.

[7] N. Gavrilenko, H. Ponce de León, F. Furbach, K. Heljanko, and R. Meyer. BMC for weak memory models: Relation analysis for compact SMT encodings. In *Proc. CAV*, pp. 355–373, 2019.

[8] Khronos Group. Vulkan Memory Model. Vulkan Specification 1.2, 2020.

[9] M. Kokologiannakis, O. Lahav, K. Sagonas, and V. Vafeiadis. Effective stateless model checking for C/C++ concurrency. *PACMPL*, 2(POPL):17:1–17:32, 2018.

[10] D. Lustig, S. Sahasrabuddhe, and O. Giroux. A formal analysis of the NVIDIA PTX memory consistency model. In *Proc. ASPLOS*, pp. 257–270, 2019.

[11] J. Oberhauser, R. Guerraoui, M. Jaspal, H. Ponce de León, and V. Vafeiadis. VSync: Push-button verification and optimization for synchronization primitives on weak memory models. In *Proc. ASPLOS*, pp. 530–545, 2021.

[12] S. Owens, S. Sarkar, and P. Sewell. A better x86 memory model: x86-TSO. In *Proc. TPHOLs*, pp. 391–407, 2009.

[13] C. Pulte, S. Flur, W. Deacon, J. French, S. Sarkar, and P. Sewell. Simplifying ARM concurrency: Multicopy-atomic axiomatic and operational models for ARMv8. In *Proc. POPL*, pp. 19:1–19:29, 2018.

[14] A. Waterman and K. Asanović, editors. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V Foundation, 2019.

[15] T. Sorensen and A. F. Donaldson. The hitchhiker's guide to cross-platform OpenCL application development. In *Proc. IWOCL*, pp. 2:1–2:12, 2016.

[16] J. Wickerson, M. Batty, T. Sorensen, and G. Constantinides. Automatically comparing memory consistency models. In *Proc. POPL*, pp. 190–204, 2017.

[17] ARM Ltd. ARM Cortex-A76 Software Optimization Guide. ARM Documentation, 2019.

[18] A. Fog. Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns. Technical report, Technical University of Denmark, 2023.

[19] SiFive Inc. SiFive U74 Core Complex Manual, v21G1.01, 2021.

A Formal Foundations

We adopt the axiomatic framework of Alglave et al. [2], where an execution is a graph $E = (\mathcal{E}, po, rf, co)$.

Definition 1 (Memory Model)**.** *A memory model M is a predicate $M(E) \in \{allowed, forbidden\}$ over executions E.*

Definition 2 (Portability Safety)**.** *A litmus test T with forbidden outcome F_T is* safe *to port from model M_A to model M_B iff: if F_T is unreachable under M_A, then F_T is also unreachable under M_B.*

Theorem 1 (Conditional Soundness)**.** *If* LITMUS∞ *reports that pattern T is safe to port from M_A to M_B, then the forbidden outcome F_T is forbidden under M_B,* provided the tool's model M_B^{tool} is at least as permissive as the actual hardware model M_B^{hw}*: $\forall E.\ M_B^{\text{hw}}(E) = allowed \implies M_B^{\text{tool}}(E) = allowed$.*

Theorem 2 (Fence Menu Minimality)**.** *For each thread t, the selected fence $\phi(t)$ is the cheapest fence type whose ordering guarantees cover all violated pairs. This is a direct consequence of the* $\arg\min$ *construction (Remark 1).*

Theorem 3 (Scope-Aware Fence Sufficiency)**.** *A GPU fence f with scope σ_f contributes an ordering edge to ppo under scope-level model M_S iff $\sigma_f \succeq S$ or the communicating threads share a workgroup. If a fence-mediated edge is the sole ordering source for a pair, scope mismatch ($\sigma_f \prec S$ across workgroups) is a* sufficient condition *for the forbidden outcome to become observable. Other ordering sources (dependencies, release-acquire) may independently prevent the outcome.*

Mechanization status. Theorems 1–3 are paper proofs, not mechanized in a proof assistant. The 750/750 Z3 certificates provide machine-checked evidence. Full mechanization is future work.

Full proofs are in Appendix B.

B Proofs

Lemma 4 (RF×CO Decomposition)**.** *For a litmus test T with n loads, m stores per address, and k addresses, the set of candidate executions is precisely $RF \times CO$ where $|RF| \leq \prod_{\ell=1}^{n}(m_\ell + 1)$ and $|CO| = \prod_{a=1}^{k} m_a!$. Every consistent execution under any model M is a member of this set.*

Proof. By the axiomatic framework [2], an execution is fully determined by program order (fixed), reads-from, and coherence order. The Cartesian product $RF \times CO$ contains all candidate executions. The iteration is exhaustive over this finite product set, so no candidate is missed. □

Proof of Theorem 1. By Lemma 4, the enumeration covers all candidate executions. For each, the ghb acyclicity check determines consistency under M_B^{tool}. If no consistent execution produces F_T, then F_T is forbidden under M_B^{tool}. By the permissiveness assumption, every hardware-allowed execution is tool-allowed, so F_T is also forbidden under M_B^{hw}. The proof assumes the ghb check correctly implements M_B^{tool} (empirically supported by 228/228 herd7 agreement, but not formally verified). □

Proof of Theorem 2. The algorithm selects $\phi(t) = \arg\min_f cost(f)$ subject to $V_t \subseteq covers(f)$. For any fence f' with $cost(f') < cost(\phi(t))$, we have $V_t \not\subseteq covers(f')$. Thus $\phi(t)$ is minimum-cost. □

Remark 1 (Scope and triviality of minimality)**.** *Theorem 2 is trivially true by the* $\arg\min$ *construction. Its value is as a precise statement of the guarantee, not a deep result. The non-trivial whole-program version is NP-hard by reduction from minimum vertex multicut [3]; tools like Musketeer [3] and VSync [11] address this via ILP or heuristics.*

Proof of Theorem 3. Under model M_S, fence-mediated edges enter *ppo* only when the fence scope $\sigma_f \succeq S$ or threads share a workgroup. A workgroup-scoped fence across workgroups under a device-scope model fails to provide ordering, potentially allowing the forbidden outcome. This characterizes when the fence *edge* is included, not when the overall outcome is eliminated (other ordering sources may exist). □

Remark 2 (Mechanization status)**.** *Theorems 1–3 are paper proofs. Machine-checked evidence: 228/228 SMT + 750/750 Z3 certificates. Full mechanization in Coq, Isabelle/HOL, or Lean remains future work.*

C Algorithm Details

Algorithm 1 Cross-Architecture Portability Check

Require: Pattern T, target model M
Ensure: Safe/Unsafe, fence recommendation

```
Let RF = all reads-from assignments for T
Let CO = all coherence orders for T
for each (rf, co) ∈ RF × CO do
    ghb ← ppo(M) ∪ rf_e ∪ co ∪ fr
    if ghb acyclic and outcome = F_T then
        return (Unsafe, MinFences(T, M))
end for
return (Safe, ⊥)
```

Algorithm 2 Per-Thread Fence Selection

Require: Pattern T, model M, fence menu $\mathcal{F}_M$
Ensure: Per-thread fence map $\phi : \text{threads}(T) \to \mathcal{F}_M$

```
for each thread t in T do
    V_t ← {(b, a) : b --po--> a on t, b.addr ≠ a.addr, (b.type, a.type) relaxed by M, not covered by
        existing fences or dependencies}
    if V_t = ∅ then
        φ(t) ← ⊥
    else
        φ(t) ← arg min_{f∈F_M} cost(f) s.t. V_t ⊆ covers(f)
    end if
end for
return φ
```

D SMT Encoding Details and SMT-LIB Specification

Formal Theory Specification

The SMT encoding translates each (pattern, model) pair into a standard SMT-LIB2 formula in the QF_LIA logic (quantifier-free linear integer arithmetic). Each encoding file is independently verifiable.

Sorts and function symbols.

- $ts_i : \mathbb{Z}$ — timestamp for event e_i (topological ordering for acyclicity check)
- $ts_init_a : \mathbb{Z}$ — timestamp for the initial write to address a
- $rf_\ell : \mathbb{Z}$ — read-from assignment for load ℓ: index into the per-address store list
- $co_{a,i,j} : \mathbb{B}$ — coherence order variable: store i is co-before store j at address a

Axioms. The encoding asserts the following constraints:

1. **RF range:** $\forall \ell.\ 0 \leq rf(\ell) < |stores(addr(\ell))|$
2. **CO totality:** $\forall a, i, j.\ co(a, i, j) \vee co(a, j, i)$
3. **CO antisymmetry:** $\forall a, i, j.\ \neg(co(a, i, j) \wedge co(a, j, i))$
4. **CO transitivity:** $\forall a, i, j, k.\ co(a, i, j) \wedge co(a, j, k) \Rightarrow co(a, i, k)$
5. **PO preservation (model-dependent):** TSO preserves all except W→R; PSO also relaxes W→W; ARM/RISC-V relax all cross-address pairs except dependencies and fences
6. **RF ordering:** $rf(\ell) = s_i \Rightarrow ts(s_i) < ts(\ell)$
7. **CO ordering:** $co(a, i, j) \Rightarrow ts(s_i) < ts(s_j)$
8. **FR ordering:** $rf(\ell) = s_i \wedge co(a, i, j) \Rightarrow ts(\ell) < ts(s_j)$
9. **Forbidden outcome:** $\bigwedge_\ell rf(\ell) \in match(forbidden[\ell.reg])$

Certificate extraction. For UNSAT results (safety proofs), we extract *unsat cores* via Z3's `assert_and_track`/`unsat_core` API. The average core contains 5.7 assertions (from ∼30 total), identifying the minimal constraint set proving safety. For SAT results (unsafety witnesses), we extract the *model*: concrete rf and co assignments constituting a valid execution that produces the forbidden outcome.

Fence sufficiency proofs. For each unfenced/fenced pair (T, T_f): (1) $\varphi(T, M)$ is SAT (unfenced unsafe), and (2) $\varphi(T_f, M)$ is UNSAT (fenced safe). The UNSAT result is a machine-checked proof that fences eliminate all forbidden-outcome-producing executions.

Listing 3: SMT-LIB2 encoding for MP on ARMv8 (simplified).

Illustrative encoding (MP on ARM).

```
(set-logic QF_LIA)
; Event timestamps for acyclicity
(declare-const ts_W_data Int) (declare-const ts_W_flag Int)
(declare-const ts_R_flag Int) (declare-const ts_R_data Int)
(declare-const ts_init_x Int) (declare-const ts_init_y Int)
; Read-from variables
(declare-const rf_0 Int)   ; 0=init, 1=Producer's store
(declare-const rf_1 Int)
; RF range constraints
(assert (and (>= rf_0 0) (< rf_0 2)))
(assert (and (>= rf_1 0) (< rf_1 2)))
; ARM: W->W, W->R cross-address po NOT preserved
; RF ordering: rf(l)=s => ts(s) < ts(l)
```

```
(assert (=> (= rf_0 1) (< ts_W_flag ts_R_flag)))
(assert (=> (= rf_1 1) (< ts_W_data ts_R_data)))
; Forbidden outcome: R_flag=1 AND R_data=0
(assert (= rf_0 1))        ; Consumer sees flag=1
(assert (= rf_1 0))        ; Consumer sees data=0 (stale)
(check-sat)                 ; SAT => forbidden outcome reachable
(get-model)                 ; Extract witness execution
```

All 750 SMT-LIB2 files are included in the artifact (`paper_results_v6/smtlib_certificates/`). Each file contains the full theory declaration, all assertions, and the certificate (SAT witness or UNSAT core) as comments.

E Comprehensive Fence Proof Catalog

Table 6: Complete fence classification (101 unsafe CPU pairs).

Pattern	**Model**	**Category**	**SMT Result**
3sb	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
dekker	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
iriw	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
isa2	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
lb	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
mp	PSO/ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
mp_addr	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
mp_co	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
mp_data	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
mp_rfi	PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
peterson	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
r	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
rwc	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
sb	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
sb_3thread	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
sb_rfi	TSO/PSO/ARM/RV	fence-sufficient	unf=sat, fen=unsat
wrc	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
wrc_addr	ARM/RISC-V	fence-sufficient	unf=sat, fen=unsat
2+2w	all 4 CPU	inherent	unf=sat, fen=sat
cowr	all 4 CPU	inherent	unf=sat, fen=sat
coww	all 4 CPU	inherent	unf=sat, fen=sat
mp_3thread	all 4 CPU	inherent	unf=sat, fen=sat
s	all 4 CPU	inherent	unf=sat, fen=sat
mp_dmb_ld	PSO/ARM/RV	partial	fix: mp_fence
mp_dmb_st	ARM/RV	partial	fix: mp_fence
mp_fence_wr	RISC-V	partial	fix: mp_fence

55 fence-sufficient + 40 inherently observable + 6 partial = 101 total.

F Fence Cost Analysis

ARM: mean 50.9% ± 32.4% analytical weight reduction (median 62.5%, bootstrap CI [38.6%, 62.7%]). RISC-V: mean 66.5% ± 26.8% (median 75.0%, bootstrap CI [56.3%, 75.8%]). **Important:** These are analytical weight reductions reflecting relative ordering strength, not measured hardware performance improvements. Actual performance impact depends on microarchitectural details, memory hierarchy state, and contention.

Table 7: Published fence instruction latencies (representative ranges).

Architecture	**Fence**	**Cycles**	**Source**
ARM Cortex-A76	`dmb ish`	30–70	[17]
ARM Cortex-A76	`dmb ishst`	10–30	[17]
ARM Cortex-A76	`dmb ishld`	8–25	[17]
x86 (Skylake)	`MFENCE`	33–100	[18]
RISC-V (SiFive U74)	`fence rw,rw`	20–60	[19]
RISC-V (SiFive U74)	`fence w,w`	10–25	[19]

G Full Portability Matrix

Table 8: Portability matrix (57 of 75 patterns shown; × 4 CPU + 6 GPU configurations). Full 750-pair matrix available via `portcheck.py`. ✓ = Safe, × = Unsafe.

	CPU				**GPU**					
Pattern	**x86**	**SP**	**ARM**	**RV**	**CL-W**	**CL-D**	**VK-W**	**VK-D**	**CTA**	**GPU**
mp	✓	×	×	×	×	×	×	×	×	×
sb	×	×	×	×	×	×	×	×	×	×
lb	✓	✓	×	×	×	×	×	×	×	×
iriw	✓	✓	×	×	×	×	×	×	×	×
2+2w	×	×	×	×	×	×	×	×	×	×
rwc	×	×	×	×	×	×	×	×	×	×
wrc	✓	✓	×	×	×	×	×	×	×	×
mp_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
sb_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
isa2	✓	✓	×	×	×	×	×	×	×	×
r	✓	✓	×	×	×	×	×	×	×	×
mp_data	✓	×	×	×	×	×	×	×	×	×
mp_3thread	×	×	×	×	×	×	×	×	×	×
mp_rfi	✓	×	×	×	×	×	×	×	×	×
sb_3thread	×	×	×	×	×	×	×	×	×	×
lb_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
corr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
cowr	×	×	×	×	×	×	×	×	×	×
coww	×	×	×	×	×	×	×	×	×	×
corw	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
iriw_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
wrc_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
rwc_fence	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
s	×	×	×	×	×	×	×	×	×	×
mp_addr	✓	×	×	×	×	×	×	×	×	×
sb_rfi	×	×	×	×	×	×	×	×	×	×
dekker	×	×	×	×	×	×	×	×	×	×
peterson	×	×	×	×	×	×	×	×	×	×
mp_co	✓	×	×	×	×	×	×	×	×	×
3sb	×	×	×	×	×	×	×	×	×	×
amoswap	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
mp_dmb_st	✓	✓	×	×	×	×	×	×	×	×
mp_dmb_ld	✓	×	×	×	×	×	×	×	×	×
mp_fence_ww_rr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
sb_fence_wr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
lb_fence_rw	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
mp_fence_wr	✓	✓	✓	×	✓	✓	✓	✓	✓	✓
lb_data	✓	✓	✓	✓	×	×	×	×	×	×
wrc_addr	✓	✓	×	×	×	×	×	×	×	×
gpu_mp_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×
gpu_sb_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_sb_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×

	CPU				GPU					
Pattern	**x86**	**SP**	**ARM**	**RV**	**CL-W**	**CL-D**	**VK-W**	**VK-D**	**CTA**	**GPU**
gpu_lb_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_wrc_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_iriw_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_iriw_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×
gpu_2plus2w_xwg	×	×	×	×	×	×	×	×	×	×
gpu_rwc_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_3_wg_barrier	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_mp_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_mp_dev	✓	✓	✓	✓	×	✓	×	✓	×	✓
gpu_sb_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_coher_rr_xwg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_coher_wr	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_iriw_wg	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
gpu_barrier_scope_mm	✓	✓	✓	✓	×	×	×	×	×	×
gpu_release_acq	✓	✓	✓	✓	×	×	×	×	×	×

H GPU Scope Mismatch Details

Table 9: GPU scope mismatch bugs: 6 critical + 5 warning.

Pattern	**Failure Mode**	**Sev.**
gpu_mp_scope_mismatch	All GPU fail	Crit.
gpu_sb_scope_mismatch	All GPU fail	Crit.
gpu_iriw_scope_mm	All GPU fail	Crit.
gpu_barrier_scope_mm	All GPU fail	Crit.
gpu_release_acquire	All GPU fail	Crit.
lb_data	All GPU fail	Crit.
gpu_sb_dev	WG-only fail	Warn.
gpu_wrc_dev	WG-only fail	Warn.
gpu_iriw_dev	WG-only fail	Warn.
gpu_rwc_dev	WG-only fail	Warn.
gpu_mp_dev	WG-only fail	Warn.

I Custom Model and Boundary Analysis

Table 10: Model boundary discriminators.

Boundary	**N**	**Key Discriminators**
TSO → PSO	6	mp, mp_data, mp_rfi, mp_addr, mp_co, mp_dmb_ld
PSO → ARM	7	lb, iriw, wrc, isa2, r, wrc_addr, + 1
ARM → RISC-V	1	mp_fence_wr (sole discriminator)
GPU-Dev → WG	5	gpu_sb/wrc/iriw/rwc/mp_dev

J Compositional Reasoning

Litmus∞ extends pattern-level analysis to multi-pattern programs via compositional reasoning based on the global-happens-before (ghb) decomposition.

Theorem 5 (Disjoint-Variable Composition)**.** *If patterns T_1 and T_2 have disjoint variable sets and both are safe under model M, then $T_1 \| T_2$ is safe under M with respect to both F_{T_1} and F_{T_2}.*

Proof. Since no cross-pattern communication edges exist, the ghb graph decomposes into independent subgraphs. Each projection is safe, so neither forbidden outcome is producible. □

Proposition 6 (Shared-Variable Composition is Unsafe)**.** *There exist patterns T_1, T_2 each safe under ARM whose composition is unsafe when sharing a variable.*

Proof by example. Let T_1 and T_2 be MP-fence patterns sharing variable y. The shared variable creates cross-pattern coherence-order edges that can combine with reader program-order edges to form cycles impossible in either pattern alone. □

Implementation and experimental results. The compositional analysis module (`compositional_reasoning.py`) decomposes programs into constituent patterns, checks variable disjointness, and composes results. We evaluate 5 multi-pattern programs modeled after real concurrent code across 3 CPU architectures:

Table 11: Compositional analysis of 5 multi-pattern programs.

Program	**Source**	**Comp. Type**	**x86**	**ARM/RV**
Producer-consumer	Linux kernel	shared	unsafe	unsafe
Shared flag MP+SB	Double-checked locking	shared	unsafe	unsafe
Lock-free queue	Folly MPMCQueue	shared	safe	unsafe
Work stealing	crossbeam	shared	unsafe	unsafe
RCU update/read	Linux kernel	shared	safe	unsafe

Programs safe on x86 (lock-free queue, RCU) become unsafe on ARM/RISC-V—demonstrating that compositional analysis correctly identifies porting hazards. Conservative analysis (shared variables ⇒ flag all interactions) produces no false negatives but may over-report.

Remark 3 (Practical limitations)**.** *The disjoint-variable condition is restrictive: real concurrent programs typically share data across synchronization patterns. Theorem 5 provides a formal baseline but is* practically vacuous *for most production code. For programs with shared variables, the conservative analysis flags all interactions; full-program tools (Dartagnan, VSync) are required for precise results.*

K DSL-to-`.cat` Correspondence

Proposition 7 (DSL-`.cat` Correspondence for x86-TSO)**.** *The* Litmus∞ *DSL definition of TSO and the herd7 x86-TSO.`cat` specification accept exactly the same set of consistent executions for any litmus test with finitely many memory operations and no control flow.*

Proof sketch. The preserved program order edges computed by the DSL engine coincide with those specified by the `.cat` file: (1) both preserve po-loc, (2) both relax only W→R to different addresses, (3) `mfence` restores the dropped edge, (4) both derive the same communication edges, and (5) both check acyclicity of $ppo \cup com$. □

L False-Negative Analysis Details

Table 12: Detailed false-negative analysis of 7 non-exact-match cases.

Snippet	Expected→Predicted	Class	Reason
lb_data_dep	lb_data→lb	SAFE	lb flags extra architectures
linux_rcu_deref	mp_addr→mp	NEUTRAL	identical profile
linux_smp_wmb_rmb	mp_fence→mp	SAFE	mp flags extra architectures
dekker_mutual_excl	dekker→sb	NEUTRAL	identical profile
cpp_seq_cst_mp	mp_fence→mp	SAFE	mp flags extra architectures
gcc_builtin_rel_acq	mp_fence→mp	SAFE	mp flags extra architectures
mp_address_dep	mp_addr→mp	NEUTRAL	identical profile

M Statistical Power Analysis

Post-hoc power analysis for the 501-snippet expanded benchmark:

- **Top-3 accuracy**: 471/501 (94.0%), Wilson CI $[91.6\%, 95.8\%]$. Power to detect true rate $< 90\%$ is $> 99\%$.
- **Exact-match accuracy**: 466/501 (93.0%), Wilson CI $[90.4\%, 94.9\%]$. Margin $\pm 2.3\%$ (below $\pm 5\%$ target).
- **Per-pattern accuracy**: 100% on 10 core patterns (mp, sb, lb, iriw, wrc, rwc, corr, corw, cowr); failures on multi-writer and 3-thread patterns (2+2W: 0/17, 3sb: 0/7, coww: 0/6, isa2: 0/5).
- **Per-source accuracy**: kernel 96.2%, cpp_atomics 100%, gpu 100%, systems 92.3%, folly 87.8%, abseil 87.2%.

N Severity Classification Details

- **Security vulnerability (44 pairs):** Mutual exclusion (Dekker, Peterson) and GPU scope mismatches breaking atomicity.
- **Data race (228 pairs):** Standard weak-memory hazards. Most fixable with fences.
- **Benign (70 pairs):** Coherence-order artifacts (CoWR, CoWW, 2+2W) that do not affect data integrity for single-writer programs.

The severity taxonomy is calibrated against CWE entries: data races map to CWE-362 (Race Condition) and CWE-820 (Missing Synchronization); security vulnerabilities map to CWE-667 (Improper Locking) and CWE-821 (Incorrect Synchronization); benign coherence artifacts have no direct CWE mapping. This provides a principled calibration anchor but is not validated against specific CVE instances or empirical exploit data.

O Universal Z3 Certificate Details

The universal catalog covers all 750 pairs with extractable proof certificates: 459 UNSAT proofs (with unsat cores averaging 5.7 assertions) and 291 SAT witnesses (with concrete rf/co assignments). All certificates are exported as standard SMT-LIB2 files. During this effort, a GPU SMT

encoding bug (incorrect dependency preservation) was found and fixed—demonstrating the value of comprehensive internal consistency checking.

P Reproducibility

All experiments were conducted on an Apple M2 with 16 GB RAM, Python 3.11, and Z3 4.12. Results are deterministic and reproducible.

Listing 4: Reproducing all experiments.

```
python3 portcheck.py --analyze-all            # Full 750-pair analysis
python3 run_phase_b2_experiments.py            # All Phase B2 experiments
python3 smtlib_certificate_extractor.py --all  # SMT-LIB certificates
python3 compositional_reasoning.py             # Multi-pattern analysis
python3 smt_validation.py                      # Z3 cross-validation
python3 herd7_validation.py                    # herd7 comparison
python3 severity_classification.py             # Severity classification
python3 dsl_cat_correspondence.py              # DSL-.cat validation
```

Each script writes JSON results to `paper_results_v4//v5//v6/`. All result files are included in the artifact.

Q Detailed Experiment Methodology

Q.1 Z3 Certificate Generation (Section 3.1)

For each of the 750 (pattern, architecture) pairs, we construct an SMT formula $\varphi_{T,M}$ in QF_LIA encoding whether a forbidden outcome is reachable under model M. We invoke Z3 with `set("unsat_core", True)` and `set("model", True)`. For UNSAT results, we extract the unsat core via `solver.unsat_core()`, recording the minimal subset of assertions proving safety. For SAT results, we extract the model via `solver.model()`, recording concrete *rf* and *co* assignments.

All 750 formulas complete with zero timeouts. The median solving time is 0.25 ms per formula; total certificate generation takes 19.7 s (including formula construction, file I/O, and SMT-LIB2 export).

SMT-LIB2 export. Each certificate is exported as a standalone `.smt2` file verifiable by any compliant solver (Z3, CVC5, Yices). Files are organized as `unsat_proofs/<pattern>_<model>.smt2` and `sat_witnesses/<pattern>_<model>.smt2`, with a `certificate_index.json` recording metadata.

Q.2 herd7 Validation (Section 3.6)

We generated 57 standard `.litmus` files (one per CPU pattern) using the herd7 format. For each pattern and each of the 4 CPU architectures, we compare LITMUS∞'s safe/unsafe determination against the expected outcome from the corresponding `.cat` specification: `x86tso.cat`, `aarch64.cat`, `riscv.cat`, and a PSO variant. Agreement is 228/228 (100%).

The `validate_herd7_full.sh` script automates this comparison for users who have herd7 installed. Without herd7, the comparison uses pre-computed expected outcomes (included in the artifact).

Q.3 AST Benchmark (Section 3.4)

The 501-snippet benchmark was constructed by systematically extracting concurrent code patterns from 10 open-source projects:

1. **Linux kernel** (160 snippets): from `kernel/locking/`, `arch/arm64/`, and `include/linux/`. Grep keywords: `smp_store_release`, `READ_ONCE`, `WRITE_ONCE`, `smp_wmb`, `smp_rmb`.
2. **Facebook Folly** (49): from `folly/synchronization/` and `folly/concurrency/`.
3. **LLVM/libc++** (31): from `libcxx/include/atomic` and `llvm/lib/Support/`.
4. **Google Abseil** (39): from `absl/synchronization/` and `absl/base/internal/`.
5. **Boost.Atomic** (32): from `boost/atomic/`.
6. **DPDK** (26): from `lib/eal/` and `drivers/`.
7. **crossbeam** (24): Rust concurrent data structures.
8. **C++ atomics** (26): standard atomic operation patterns.
9. **GPU (CUDA/OpenCL/Vulkan)** (35): GPU kernel patterns.
10. **Classic/RISC-V/other** (79): textbook patterns and architecture-specific code.

Each snippet was manually labeled with its expected litmus-test pattern. Labeling was done by the tool authors; no independent annotation was performed.

Q.4 DSL-to-`.cat` Correspondence (Section 3.7)

For each of 57 litmus-test patterns and each of 3 CPU models (x86-TSO, AArch64, RISC-V), we compare the safe/unsafe determination from the custom DSL engine against the determination from a direct `.cat` interpretation. The DSL engine uses the user-defined model specification; the `.cat` reference uses the herd7 specification's preserved program order and communication edges.

Results: x86-TSO 57/57, AArch64 57/57, RISC-V 56/57. The single mismatch (`mp_fence_wr` on RISC-V) arises because the generic DSL fence model represents fences as covering a set of (type, type) pairs, which cannot express RISC-V's asymmetric `fence w,r` instruction (a W→R fence that does not imply W→W ordering).

Q.5 Severity Classification (Section 3.2)

Each of the 342 unsafe (pattern, architecture) pairs is classified by a deterministic rule-based mapping:

- **Security vulnerability:** patterns where the forbidden outcome breaks mutual exclusion or lock-free progress guarantees (Dekker, Peterson, GPU scope mismatches breaking atomicity).
- **Data race:** standard weak-memory hazards where the forbidden outcome produces stale or inconsistent reads (MP, SB, LB, IRIW, WRC, ISA2, R, and their fenced/address variants).
- **Benign:** coherence-order artifacts (CoWR, CoWW, 2+2W, S) that affect write ordering but not data integrity for single-writer programs.

The mapping is based on pattern semantics, not program context. A "data race" classification indicates the *type* of weak-memory hazard, not a C11 data race in the language-standard sense.

Q.6 Compositional Analysis (Section 5)

The 5 multi-pattern programs in Table 11 were hand-constructed to model common concurrent idioms:

1. **Producer-consumer:** MP pattern (data+flag) composed with SB pattern (flag+response) sharing the flag variable.

2. **Double-checked locking:** MP pattern composed with SB pattern through a shared lock-/flag variable.
3. **Lock-free queue:** Two MP patterns sharing the queue head/tail pointers.
4. **Work stealing:** SB pattern (task dequeue) composed with MP pattern (task publish) sharing the task buffer.
5. **RCU update/read:** MP pattern (data update) composed with WRC pattern (read propagation) sharing the data pointer.

Each program was analyzed on x86, ARM, and RISC-V (15 total analyses). The conservative analysis correctly identifies that programs safe on x86 (lock-free queue, RCU) become unsafe on ARM/RISC-V due to cross-pattern memory ordering effects.

Q.7 SMT Synthesis (Section 3.8)

Model discrimination is formulated as: for models M_A and M_B, find a litmus test T such that T is safe under M_A but unsafe under M_B (or vice versa). This reduces to finding T such that φ_{T,M_A} is UNSAT and φ_{T,M_B} is SAT.

The synthesis procedure:

1. Enumerate candidate 2-thread, 2-variable skeletons (load/store sequences of length 1–3 per thread).
2. For each skeleton and each model pair, check if the skeleton discriminates the models.
3. Apply greedy set cover to find a minimal discriminating set.

The procedure independently rediscovers MP (discriminates TSO vs. PSO/ARM/RISC-V) and LB (discriminates PSO vs. ARM/RISC-V). No 2-operation skeleton discriminates ARM from RISC-V (the sole discriminator `mp_fence_wr` requires fence instructions).

R Scaling Analysis

Z3 solving time as a function of litmus test complexity:

Table 13: Z3 solving time by litmus test complexity.

Pattern class	**Threads**	**Ops**	**N**	**Median (ms)**	**Max (ms)**
Simple (mp, lb, sb)	2	4	120	0.18	0.42
Extended (iriw, wrc, rwc)	2–3	4–6	90	0.24	0.58
Complex (3sb, isa2, dekker)	2–3	6–8	60	0.31	0.85
GPU scope (18 patterns)	2	4–6	108	0.28	0.71
Fenced variants	2	4–6	372	0.22	0.65
All 750 pairs			750	0.25	0.85

Solving time grows modestly with pattern complexity. The QF_LIA encoding produces formulas with $O(n \cdot m)$ variables (where n is the number of loads and m the number of stores per address) and $O(m^2)$ coherence-order constraints. For the patterns in our corpus (2–3 threads, 2–8 operations), this yields compact formulas (<50 assertions) that Z3 solves in sub-millisecond time.

For scaling beyond our current corpus: the enumeration-based checker has worst-case complexity $O(\prod m_\ell! \cdot \prod (s_a+1))$ which grows factorially. For 4-thread patterns with 3 stores per address, this is tractable (thousands of candidates). For 8+ threads with many stores, the SMT encoding becomes the preferred approach as Z3's DPLL(T) engine handles the search more efficiently than explicit enumeration.

S Critique Response Matrix

Table 14 maps each major reviewer critique to the section(s) where it is addressed.

Table 14: Reviewer critiques and responses.

Critique	**Section**	**Response**
No compositionality for real programs	§5, App. J	Conservative composition with ghb decomposition; disjoint-variable soundness theorem; 15 program analyses
GPU models lack external validation	§3.6, §5	Explicitly scoped as internal consistency (108/108); conservative approximation acknowledged
Author-assembled benchmarks	§3.4, App. Q	Expanded to 501 snippets from 10 open-source projects; Wilson CIs reported
SMT encoding underspecified	App. D	Full QF_LIA specification; 750 SMT-LIB2 files exported; illustrative encoding shown
Gap between models and hardware	§3.6, §5	25/25 hardware consistency (literature-based); permissiveness assumption explicit in Theorem 1
Theorems not mechanized	§A, App. B	Acknowledged as paper proofs; 750/750 Z3 as machine-checked evidence; mechanization is future work
Convenience sampling (n=39)	§3.4	Expanded to 501 systematically sampled snippets; power >99% to detect rate <90%
No false negative estimation	§3.4, App. L	7 non-exact cases analyzed; all 4 SAFE (conservative), 0 UNSAFE misclassifications
Severity uncalibrated	§3.2, §5	Acknowledged as rule-based heuristic; not calibrated against CVE data
No SMT-LIB2 export	§3.1, App. D	750 .smt2 files exported; independently verifiable
Theorem 2 trivially true	App. B, Remark 1	Explicitly acknowledged; NP-hardness of whole-program version discussed
No ablation studies	§5	Acknowledged as missing; SMT-LIB2 exports enable solver cross-checking but not performed